

Documentación de la funcionalidad añadida

Checkpoint - Sensor biordinario | Proyecto BlueTech

1. Descripción general

Para esta variante del checkpoint se ha incorporado soporte para un nuevo sensor llamado sensor biordinario. Este sensor se diferencia de los sensores originales porque genera dos datos por lectura: un valor numérico y un valor alfanumérico.

La ESP32 no se conecta directamente a MySQL. La placa envía los datos al servidor Flask mediante una petición HTTP GET y, posteriormente, Flask realiza la inserción en la base de datos del proyecto.

Flujo de comunicación implementado

ESP32 / Arduino -> DatabaseInsert() -> Flask / app.py -> MySQL -> medicion_biordinario

2. Cambios realizados en la base de datos

Se ha creado una tabla nueva llamada medicion_biordinario para almacenar los dos valores generados por el sensor. Se ha optado por crear una tabla específica para no modificar la tabla medicion original ni afectar al funcionamiento existente de temperatura, humedad y gas.

Campo	Descripción
id_medicion_biordinario	Identificador único de cada medición.
fecha_hora	Fecha y hora en la que se registra la lectura.
valor_numerico	Dato numérico generado por el sensor biordinario.
valor_alfanumerico	Dato textual generado por el sensor biordinario.
fk_id_sensor	Clave foránea que asocia la medición con el sensor registrado en la tabla sensor.

Script SQL añadido

```
-- CHECKPOINT - SENSOR BIORDINARIO
CREATE TABLE IF NOT EXISTS medicion_biordinario (
    id_medicion_biordinario INT AUTO_INCREMENT PRIMARY KEY,
    fecha_hora DATETIME NOT NULL,
    valor_numerico FLOAT NOT NULL,
    valor_alfanumerico VARCHAR(100) NOT NULL,
    fk_id_sensor INT NOT NULL,
    FOREIGN KEY (fk_id_sensor) REFERENCES sensor(id_sensor)
);

-- Registro del sensor Biordinario si todavía no existe.
INSERT INTO sensor
(estado, fecha_instalacion, fk_id_habitacion, fk_id_parametro, consumo, tipo_sensor)
SELECT 'Activo', '2026-05-20', 1, 1, '0.00', 'Biordinario'
WHERE NOT EXISTS (
    SELECT 1
    FROM sensor
    WHERE tipo_sensor = 'Biordinario'
);
```

Nota: en la base de datos local utilizada durante la prueba el sensor biordinario quedó registrado con id_sensor = 21. Ese identificador se usa en app.py para asociar las mediciones al sensor correspondiente.

3. Cambios realizados en app.py

En app.py se ha añadido una nueva ruta Flask llamada /biordinario. Esta ruta recibe los parámetros enviados desde Arduino, valida los datos y llama a una función de inserción específica para guardar la medición en MySQL.

Constante añadida

```
# ID del nuevo sensor biordinario registrado en la tabla sensor.  
ID_SENSOR_BIORDINARIO = 21
```

Función de inserción añadida

```
def guardar_medicion_biordinario(valor_numerico, valor_alfanumerico):  
    conexion = obtener_conexion()  
  
    if conexion:  
        try:  
            cursor = conexion.cursor()  
            query = """  
                INSERT INTO medicion_biordinario  
                (fecha_hora, valor_numerico, valor_alfanumerico, fk_id_sensor)  
                VALUES (NOW(), %s, %s, %s)  
            """  
            cursor.execute(query, (  
                valor_numerico,  
                valor_alfanumerico,  
                ID_SENSOR_BIORDINARIO  
            ))  
            conexion.commit()  
            cursor.close()  
            conexion.close()  
        except Exception as e:  
            print(f"Error al insertar medición biordinaria: {e}")
```

Ruta Flask añadida

```
@app.route("/biordinario", methods=["GET"])  
def recibir_biordinario():  
    valor_num = request.args.get("num")  
    valor_texto = request.args.get("txt")  
  
    if not valor_num or not valor_texto:  
        return "Faltan datos del sensor biordinario", 400  
  
    try:  
        valor_num = float(valor_num)  
    except ValueError:  
        return "El valor numerico no es valido", 400  
  
    guardar_medicion_biordinario(valor_num, valor_texto)  
    print(f"Biordinario recibido -> num: {valor_num}, txt: {valor_texto}")  
    return "OK biordinario", 200
```

4. Cambios realizados en Arduino

En el fichero de Arduino se ha añadido una nueva URL para la ruta /biordinario y se ha implementado el método DatabaseInsert(), solicitado en el checkpoint. Este método recibe el valor numérico y el valor alfanumérico del sensor, construye una URL con ambos datos y los envía a Flask mediante HTTP GET.

Ruta añadida en Arduino

```
const char* serverBiordinario = "http://172.20.10.4:5000/biordinario";
```

Método DatabaseInsert()

```
void DatabaseInsert(float valorNumerico, String valorAlfanumerico) {
  if (WiFi.status() == WL_CONNECTED) {
    String url = String(serverBiordinario)
      + "?num=" + String(valorNumerico)
      + "&txt=" + valorAlfanumerico;

    HTTPClient http;
    http.begin(url);
    int code = http.GET();

    Serial.print("Respuesta Flask biordinario: ");
    Serial.println(code);

    http.end();
  } else {
    Serial.println("Error: WiFi no conectado. No se puede enviar biordinario.");
  }
}
```

Simulación del sensor biordinario en loop()

```
float datoNumericoBiordinario = random(10, 100);
String datoAlfanumericoBiordinario;

if (datoNumericoBiordinario < 40) {
  datoAlfanumericoBiordinario = "BAJO";
} else if (datoNumericoBiordinario < 70) {
  datoAlfanumericoBiordinario = "NORMAL";
} else {
  datoAlfanumericoBiordinario = "ALTO";
}

DatabaseInsert(datoNumericoBiordinario, datoAlfanumericoBiordinario);
```

5. Instrucciones de ejecución y prueba

5.1 Preparar la base de datos

1. Abrir XAMPP e iniciar el servicio MySQL.
2. Entrar en phpMyAdmin.
3. Importar el script actualizado Database/BluetechBBDD.sql.
4. Comprobar que existe la tabla medicion_biordinario.
5. Comprobar que existe el sensor Biordinario en la tabla sensor.

```
SHOW TABLES;
```

```
SELECT *
FROM sensor
WHERE tipo_sensor = 'Biordinario';
```

5.2 Ejecutar Flask

6. Abrir una terminal en la carpeta raíz del proyecto.
7. Ejecutar `app.py`.
8. Dejar la terminal abierta mientras se realizan las pruebas.

```
python app.py
```

5.3 Probar sin Arduino

Antes de subir el código a la ESP32 se puede comprobar la ruta desde el navegador:

```
http://127.0.0.1:5000/biordinario?num=55.5&txt=NORMAL
```

Si todo funciona correctamente, Flask debe devolver:

```
OK biordinario
```

Después se puede comprobar la inserción en MySQL:

```
SELECT *  
FROM medicion_biordinario  
ORDER BY fecha_hora DESC;
```

5.4 Configurar Arduino

9. Comprobar la IP del ordenador donde se ejecuta Flask usando `ipconfig`.
10. Actualizar en Arduino las rutas `serverName` y `serverBiordinario` con esa IP.
11. No usar `localhost` ni `127.0.0.1` en Arduino, porque para la ESP32 esas direcciones hacen referencia a la propia placa.
12. Subir el código a la ESP32 y abrir el monitor serie.

```
const char* serverName = "http://IP_DEL_PC:5000/datos";  
const char* serverBiordinario = "http://IP_DEL_PC:5000/biordinario";
```

En el monitor serie debe aparecer una respuesta similar a:

```
Respuesta Flask biordinario: 200
```

6. Comprobación final

La funcionalidad se considera correcta si se cumplen estas condiciones:

- Flask responde OK biordinario al probar la URL manualmente.
- Arduino muestra Respuesta Flask biordinario: 200.
- La tabla `medicion_biordinario` recibe nuevas filas.
- Cada fila contiene `fecha_hora`, `valor_numerico`, `valor_alfanumerico` y `fk_id_sensor`.

7. Explicación final

Para el checkpoint se ha añadido un nuevo sensor biordinario que genera dos datos por lectura: uno numérico y otro alfanumérico. Para almacenarlo se ha creado la tabla `medicion_biordinario`. En Arduino se ha implementado el método `DATABASEINSERT()`, que recibe ambos valores, construye una URL con los parámetros `num` y `txt` y los envía por HTTP GET a Flask. En `app.py` se ha creado la ruta `/biordinario`, que recibe esos valores, valida el dato numérico y ejecuta un INSERT en MySQL. De esta forma se mantiene el flujo ESP32 -> Flask -> base de datos.